Tula's Canteen

COMPLETE PROJECT GUIDE

Order ahead. Skip the queue. Run the canteen on data.

Everything about the project in one place: what it does, how every feature and the smart parts work, the architecture and technology, and how to present it to anyone.

Java 21 · Spring Boot 3.5 | React 19 · Tailwind CSS 4 | PostgreSQL on Neon | Live updates (SSE) · Web Push | Razorpay | Live on Render

Abhishek Raj
Developer · November 2026

tulas-canteen.onrender.com
github.com/Abhishek360007/Canteen-Management-System-

OVERVIEW

Contents

01 Project at a glance

02 The problem and the solution

03 Who uses it: roles and permissions

04 The complete workflow

05 Feature tour with screenshots

06 How the smart parts work

07 System architecture

08 The life of an order

09 Technology stack

10 Database design

11 APIs and live updates

12 Security

13 Reliability and engineering decisions

14 Testing and quality

15 Deployment and operations

16 How to present the project

17 Questions you will be asked

18 Future roadmap

19 Glossary

20 Credits

> **How to use this guide.** Read sections 1–5 to understand *what* the product does. Read 6–15 to understand *how* it is built. Use 16–17 before any demo, presentation or interview.

Live links

Live website	https://tulas-canteen.onrender.com
Demo video	https://abhishek360007.github.io/Canteen-Management-System-/demo/
Source code	https://github.com/Abhishek360007/Canteen-Management-System-
API docs	Swagger UI at `/swagger-ui.html` on the server

01 · PROJECT AT A GLANCE

A campus canteen that runs on pre-orders

Tula's Canteen is a full-stack web application for ordering food ahead on a college campus. Students order and pay from their phone, the kitchen cooks from a live order board, food is handed over against a 4-digit pickup code, and admins see sales, forecasts and student feedback in one place.

> "Pick your food and a pickup time, pay from your canteen wallet, and walk up when it's ready. No queue, no shouting order numbers, no running out of change, and the kitchen knows what to cook before the rush."

Key numbers

3
user roles: student, kitchen staff and admin

101
REST endpoints, documented in Swagger

22
database tables

60
automated tests, run on H2 and a real PostgreSQL

1
database transaction per order: plates, payment and points together

43
dishes in the one-click standard canteen menu

∞
canteens on one campus, each with its own menu and kitchen

₹0
monthly hosting cost (free tiers of Render and Neon)

What it can do

1

Order ahead

Menu by canteen with live stock, a pickup time slot, a note for the kitchen, wallet or cash payment and reward points.

2

Live kitchen board

New orders appear instantly with a chime. One tap to start cooking, one to mark ready; the student's phone updates at once.

3

Safe hand-over

Order number plus a 4-digit pickup code or QR. Wrong codes are counted and the order locks after 5 tries.

4

Money done right

Campus wallet with a full ledger, UPI top-ups approved by admins, online payment with Razorpay, refunds and rewards.

5

Smart insights

Recommendations from basket analysis, a demand forecast per dish, an ingredient list and peak-hour heatmaps.

6

Feedback loop

Dish reviews with moderation, and complaints handled in a thread that can end with a wallet refund.

02 · THE PROBLEM AND THE SOLUTION

Why this project exists

The problem

The lunch-hour queue

Hundreds of students arrive in the same 20 minutes. Most of the break is spent standing in line to order, then again to collect.

The counter is the bottleneck

Taking orders, collecting cash, giving change and calling out numbers all happen at one window, so everything waits on it.

Cooking by guesswork

The kitchen guesses how many plates to prepare. Too few and dishes sell out at 1 pm; too many and food is wasted.

Problems go unheard

Cold food or a missing item turns into a complaint in a corridor. Nobody records it, nobody fixes it, and the student never gets a refund.

The solution

1. **Order and pay before you arrive**
   Students order from their phone, choose a pickup time and pay from a campus wallet, so the counter only hands food over.
2. **The kitchen sees every order live**
   Orders land on a board in the kitchen the moment they are placed, in the order they need to be cooked.
3. **Collect in seconds with a code**
   The pickup pass shows the order number, a 4-digit code and a QR; staff check it and hand over. No calling out names.
4. **Decisions from data**
   Forecasts say how many plates to make and which ingredients to buy; reviews and complaints say what to fix.

The impact

Before	With Tula's Canteen
Two queues: one to order, one to collect	No ordering queue; collection takes seconds with a pickup code
Cash and change at the window	Wallet, UPI and online payment, with a ledger of every rupee
"Sold out" at the peak, waste at closing	Live stock badges, low-stock alerts and a per-dish forecast for tomorrow
Complaints lost in conversation	Recorded complaints, a reply thread and refunds straight to the wallet
No idea what sells when	Sales dashboard, peak-hour heatmap, best and least sellers

03 · WHO USES IT

Roles and permissions

Every person logs in with their own account. What they see and can do depends on their role, and the server enforces it on every request; hiding a button is never the only protection.

Role	What they do	Main screens
Student	Browse the menu of a chosen canteen, order ahead, pay from the wallet or at the counter, track the order live, collect with the pickup code, review dishes, report problems, earn and spend reward points.	Menu, Cart, Orders and pickup pass, Wallet, Rewards, Notifications, Account
Kitchen staff	Work one canteen: cook from the live board, hand over food after checking the code, switch dishes on or off, set plates left, mark the canteen Open / Busy / Closed, check the day's cash.	Live orders, Hand over, Stock, Summary
Admin	Everything above for every canteen, plus menus, users and wallets, UPI top-up approvals, online payments, complaints and reviews, forecasts and insights, announcements and the activity log.	Dashboard, Orders, Menu, Users, Complaints, Reviews, Insights, Canteens, Payments, Activity

> **Staff are pinned to their canteen.** A cook at the Food Court can't see, change or hand over orders from the main canteen. The server checks the canteen of every order and dish against the staff member's canteen.

Accounts

- Students sign up themselves with name, email, roll number and password.
- Admins create kitchen staff logins and assign each one to a canteen.
- The first admin account is created when the server first starts, from `ADMIN_EMAIL` and `ADMIN_PASSWORD`. If no password is set, a random one is generated and printed once in the server log.
- Admins can block a user (logged out immediately), reset a forgotten password with a temporary one, or log someone out on all devices.

04 · THE COMPLETE WORKFLOW

From hungry student to empty plate

This is the journey of one order through the application. Each step maps to a screen you can show in a demo.

1. **Sign in and choose a canteen**
   The student logs in (a signed JWT identifies them on every request) and picks where they will collect.

2. **Build the order**
   Today's menu shows prices, ratings, veg marks and "only N left". The cart suggests add-ons that people often order together.

3. **Choose a time and pay**
   Pickup as soon as possible or in a 15-minute slot, an optional note, wallet or cash, and reward points if they have them.

4. **Order placed in one transaction**
   The rules are checked, plates are reserved, the wallet is charged and points are used, all together or not at all.

5. **The kitchen cooks**
   The order appears on the live board with a chime. Staff tap **Start cooking**, then **Mark ready**; the student's pass updates instantly and their phone buzzes.

6. **Hand-over**
   At the counter, staff scan the QR or type the order number and the 4-digit code. The order is marked collected and reward points are added.

7. **Feedback**
   The student rates each dish and, if something was wrong, reports a problem within 48 hours.

8. **Resolution**
   An admin replies in the complaint thread and resolves it, refunding a dish or the whole order to the wallet if needed.

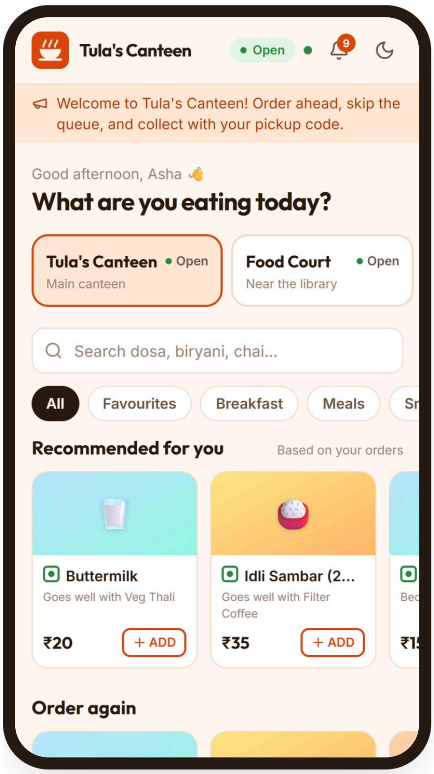
9. **Plan tomorrow**
   The admin dashboard and forecast show what sold, when, and how many plates of each dish to prepare tomorrow.

What if the student doesn't come? A background job marks ready food that isn't collected within the canteen's hold time as *not collected* and puts the plates back on sale. Unfinished orders from earlier days are cancelled and refunded automatically, and stock resets every morning at 5 AM.

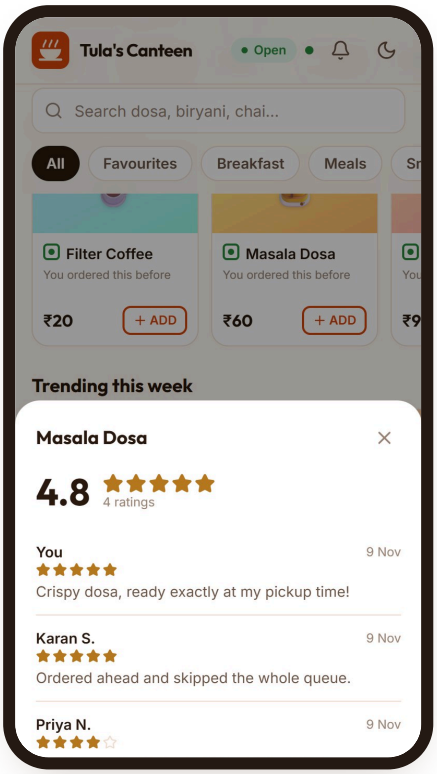
05 · FEATURE TOUR

The product, screen by screen

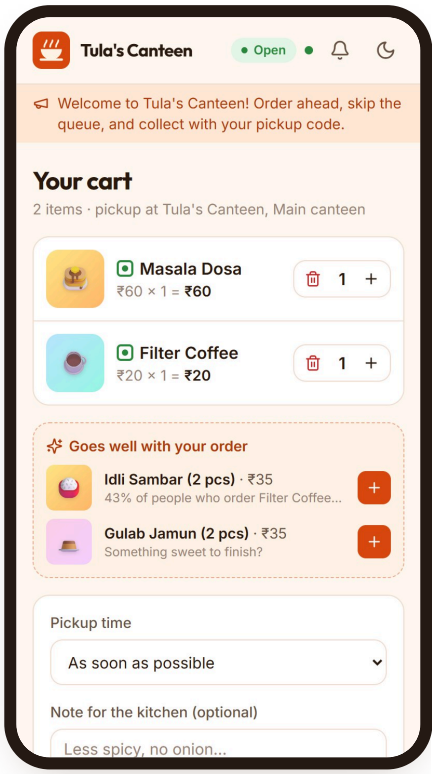
For students (on the phone)



Menu. Pick a canteen; recommendations for you.



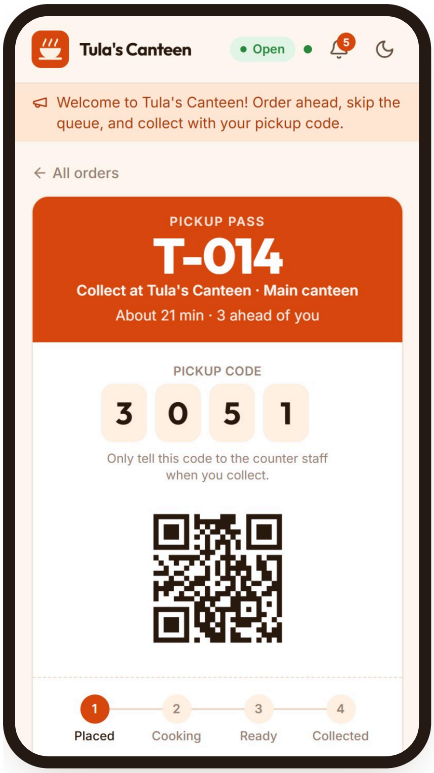
Dish reviews. Tap a rating to read them.



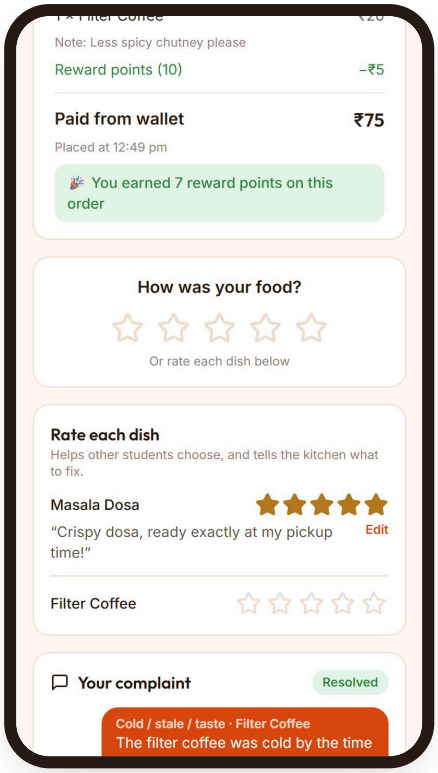
Cart. Add-ons, pickup time, wallet, points.

Menu. Dishes are grouped by category with search, a veg-only filter and favourites. Each dish shows its price, prep time, rating and "only N left" when stock is low; sold-out dishes offer similar alternatives. The top strip is personal: "Recommended for you", "Order again" and "Trending this week".

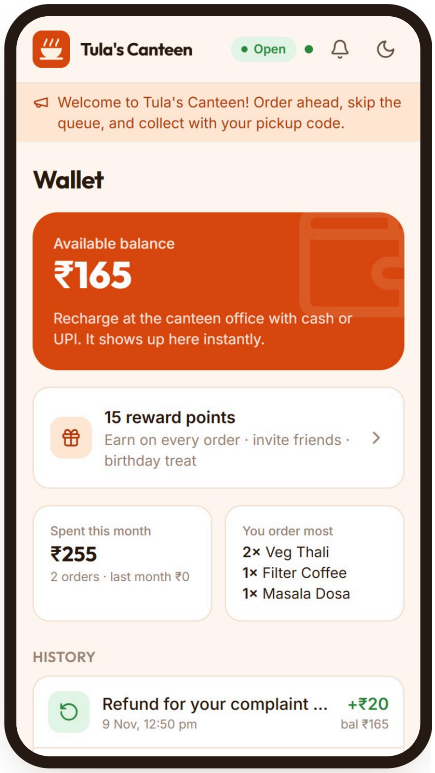
Cart. "Goes well with your order" uses real order data ("43% of people who order Filter Coffee add Idli Sambar"). The student picks a pickup slot, adds a note, chooses wallet or cash and can use reward points (10 points = ₹5, up to half the order). If the wallet is short, they can pay just the difference online.



Pickup pass. Number, code, QR, live status.

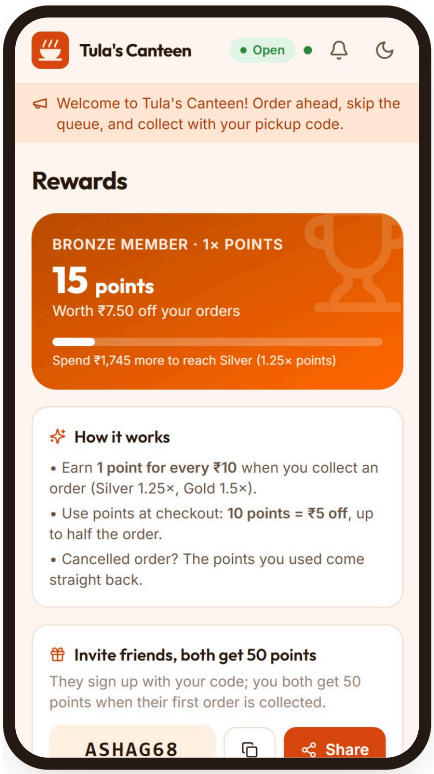


After pickup. Rate dishes, report a problem.

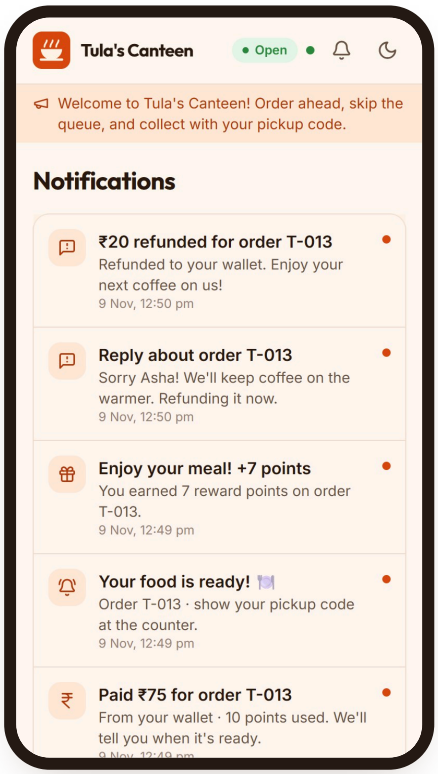


Wallet. Balance, top-ups, full history.

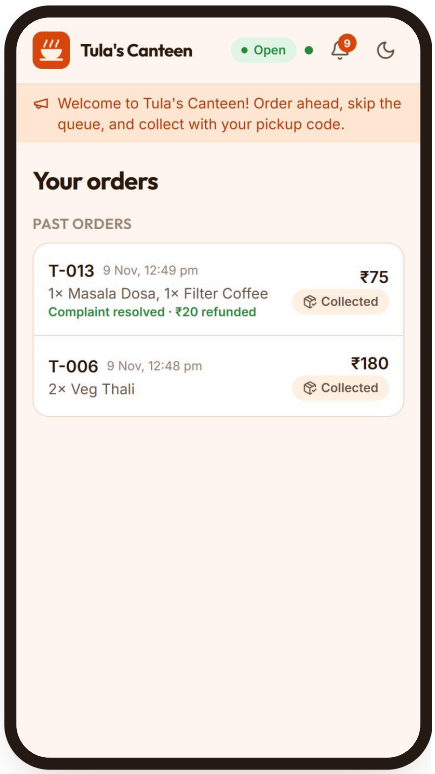
Pickup pass. The order number (e.g. `T-017`), a 4-digit pickup code and a QR, with Placed → Cooking → Ready → Collected, the queue position and the estimated wait, all updating live. Before cooking starts the student can cancel and gets an instant wallet refund.



Rewards. Points, tiers, birthday, referrals.



Notifications. In-app bell plus phone alerts.



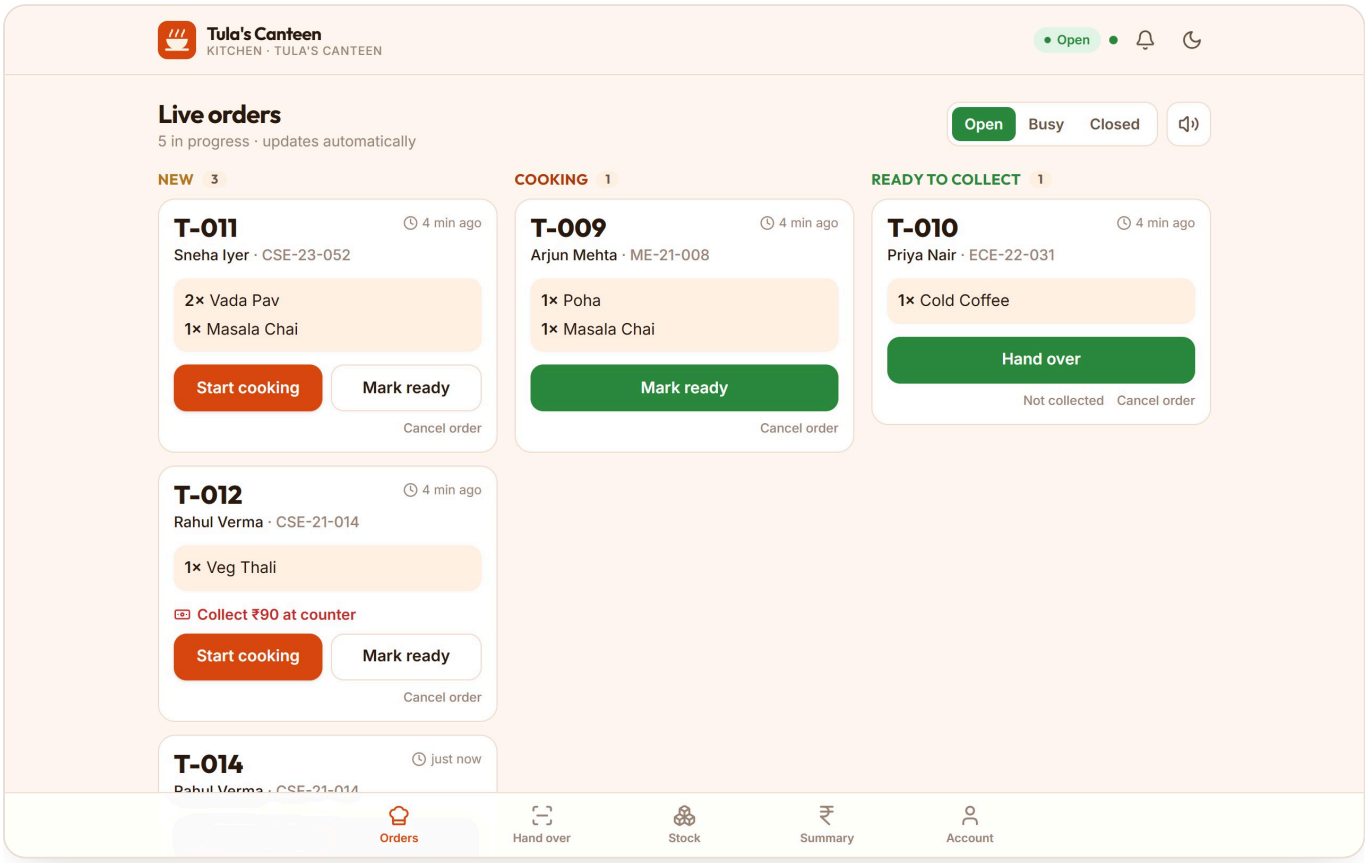
Orders. In progress and history.

Wallet and rewards. One campus wallet works at every canteen. Money comes in at the counter, by UPI (the student submits the UPI reference and an admin approves it) or online with Razorpay. Every collected order

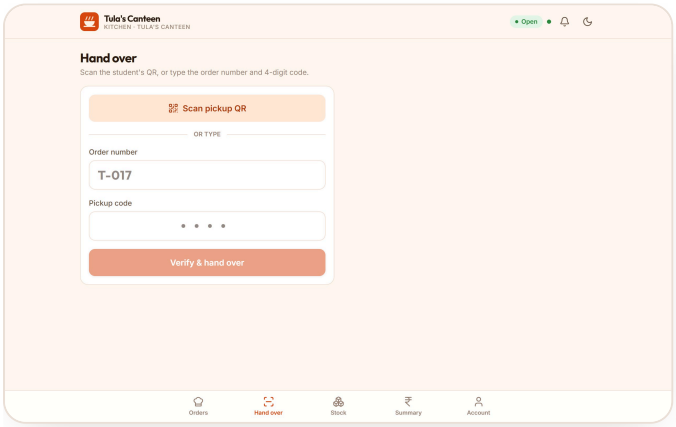
earns 1 point per ₹10; Silver and Gold tiers earn 1.25× and 1.5×, and there is a birthday bonus and 50 points each for inviting a friend.

Notifications. A bell lists every update, and browser push alerts the phone even when the site is closed: food ready, order cancelled and refunded, payment received, top-up approved, points earned, and replies to a complaint.

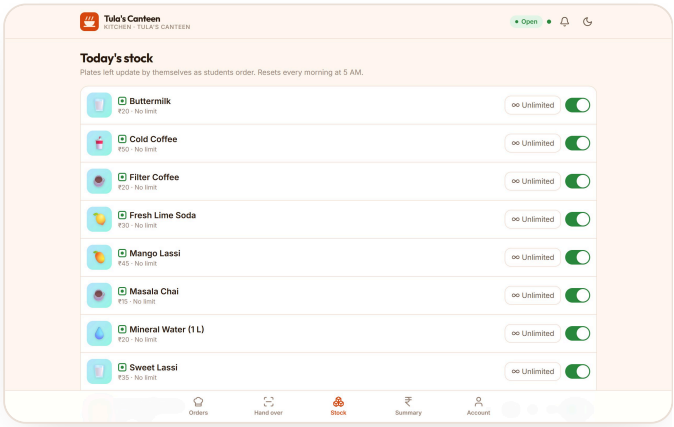
For the kitchen



Live orders. New, Cooking and Ready columns update by themselves, with a chime for every new order. Cash orders show the amount to collect; notes from the student are highlighted.



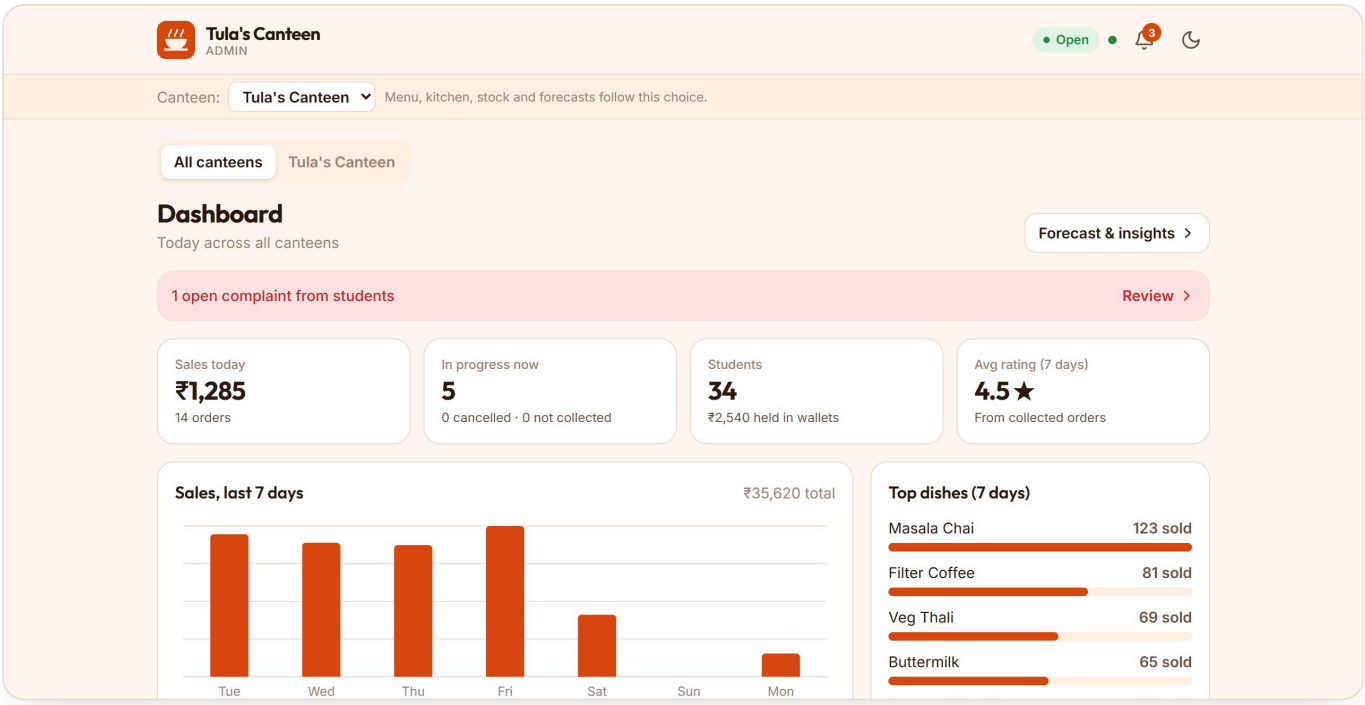
Hand over. Scan the QR or type the order number and code. After 5 wrong codes the order locks.



Stock. Plates left per dish, unlimited dishes, and on/off for today.

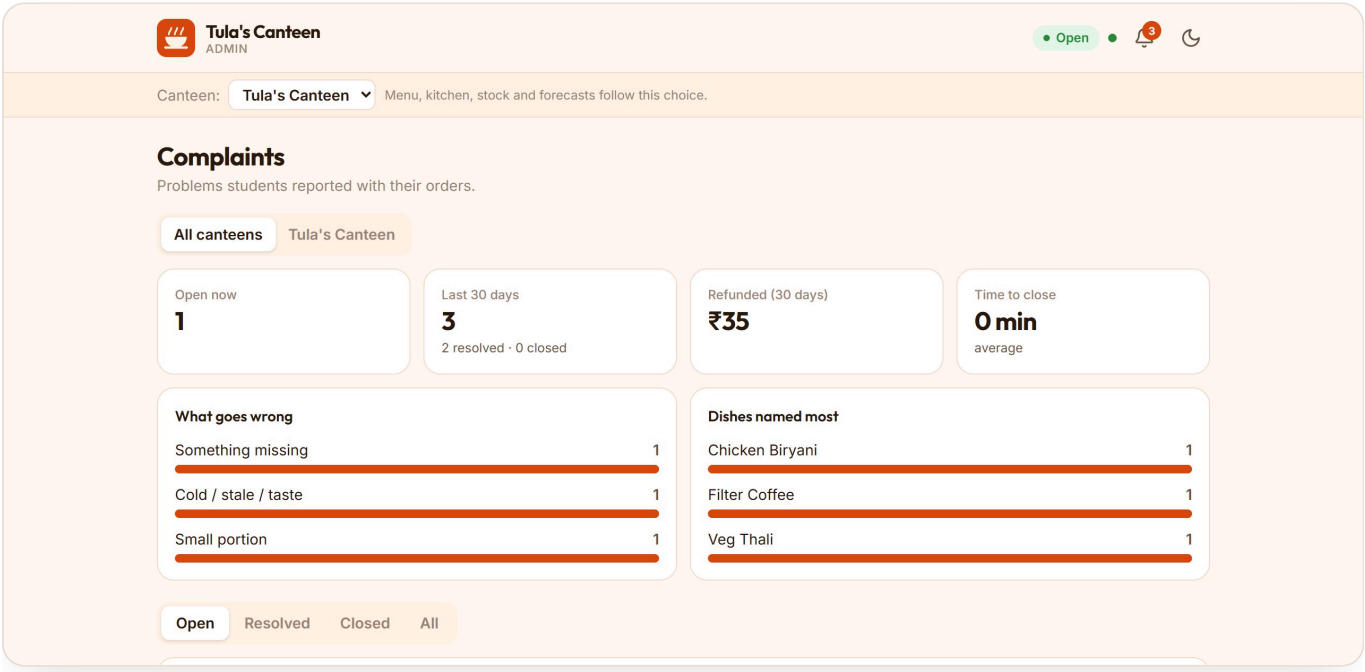
Open / Busy / Closed. One switch tells students whether the canteen is taking orders. **Day summary** shows cash collected, cash still to collect, wallet sales and items sold, plus the plan for the coming days: how many plates of each dish to prepare. The kitchen gets a phone alert when a dish is running low or sells out.

For admins

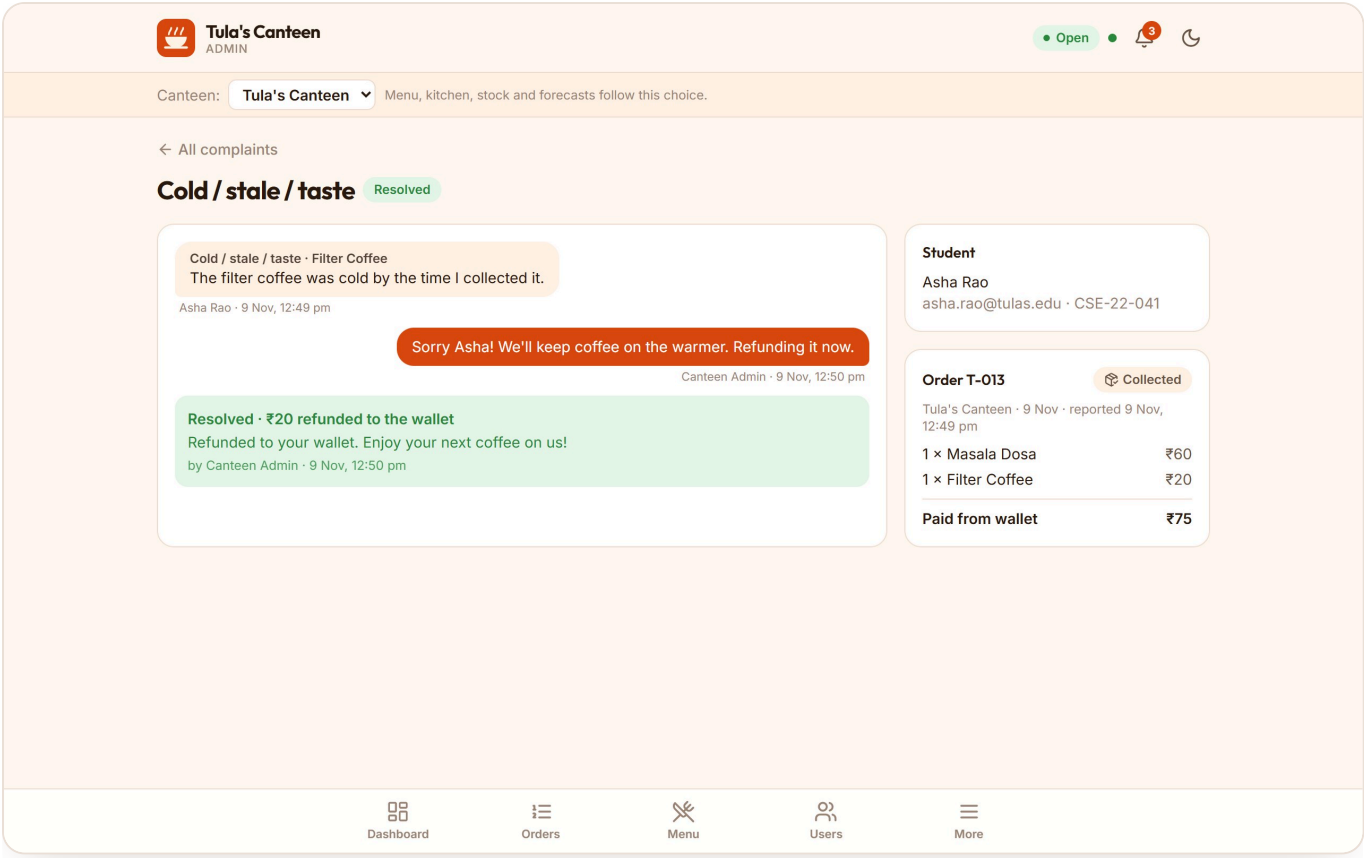


Dashboard. Today's sales and orders, orders in progress, students and the money held in wallets, average rating, the last 7 days, top dishes and the busiest hours, for one canteen or all of them. Banners flag open complaints and UPI top-ups waiting for approval.

Complaints: listen, reply, refund



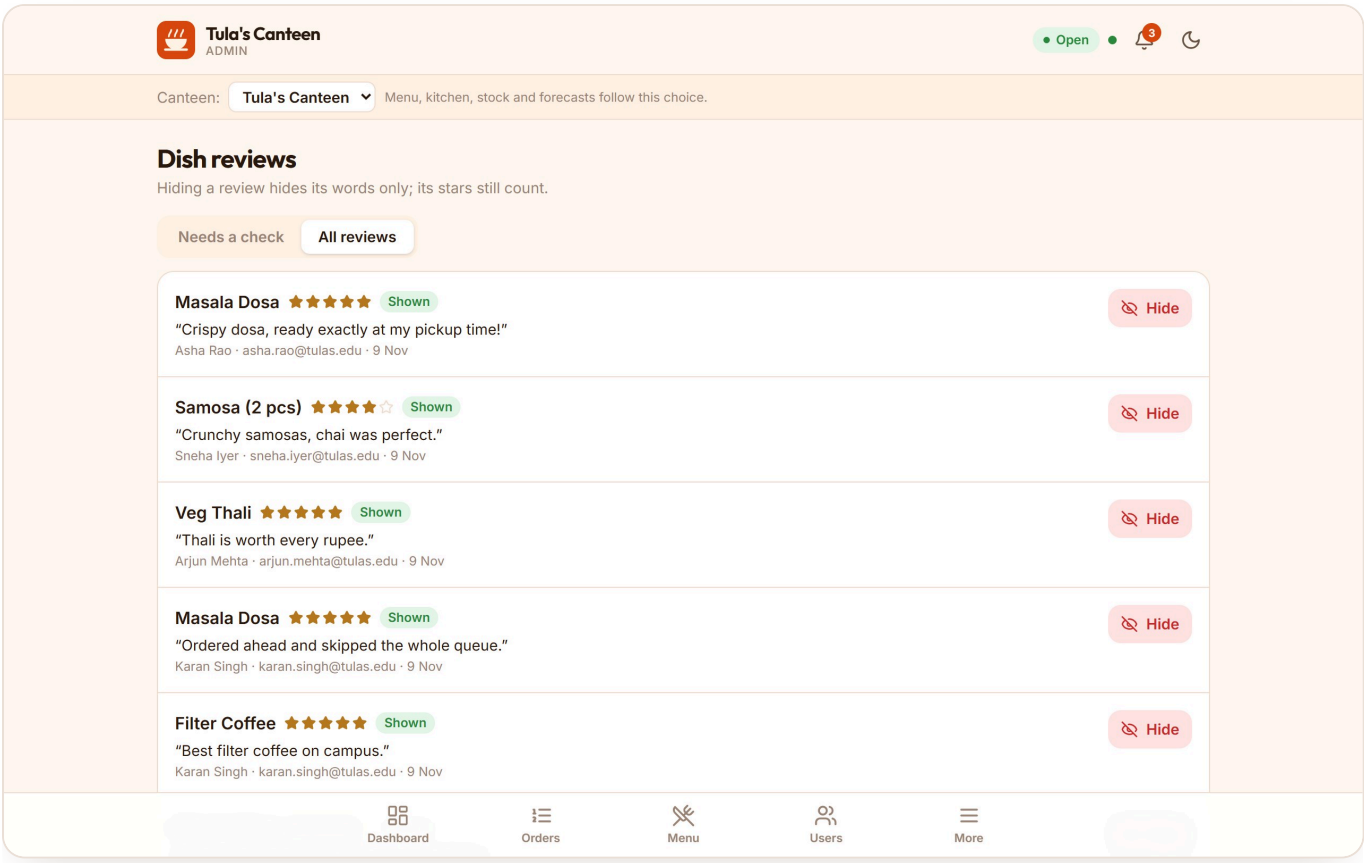
Complaints. A 30-day summary (what goes wrong, which dishes are named, refunds, average time to close) above the list of open, resolved and closed complaints.



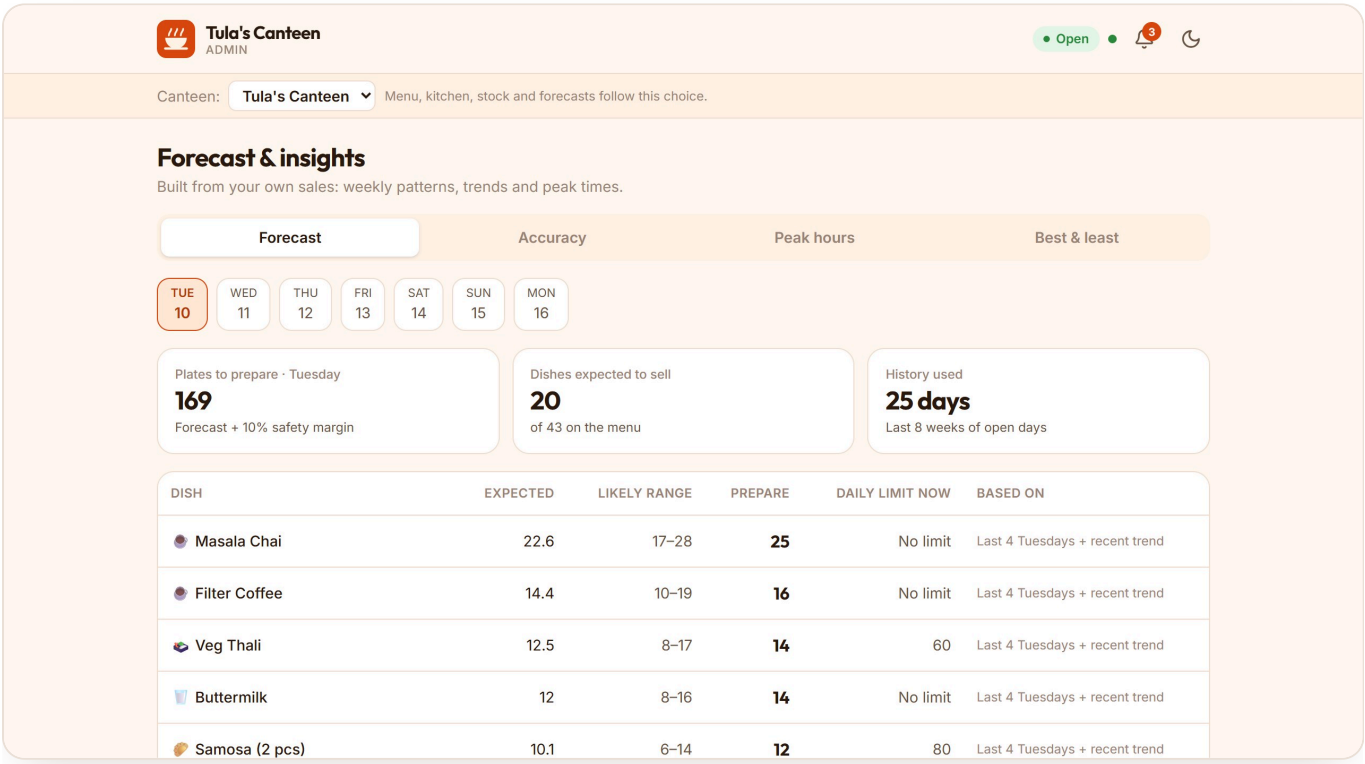
A complaint thread. The student's report, the conversation, the order and what was paid, and the outcome: here ₹20 refunded to the wallet.

Students can report a collected (or not-collected) order up to 48 hours later: they pick a category and the dishes, and describe what happened. Admins reply (with quick replies), then **resolve** it with an optional refund (one tap for a dish or the whole order, never more than the student actually paid) or **close** it with a reason the student sees.

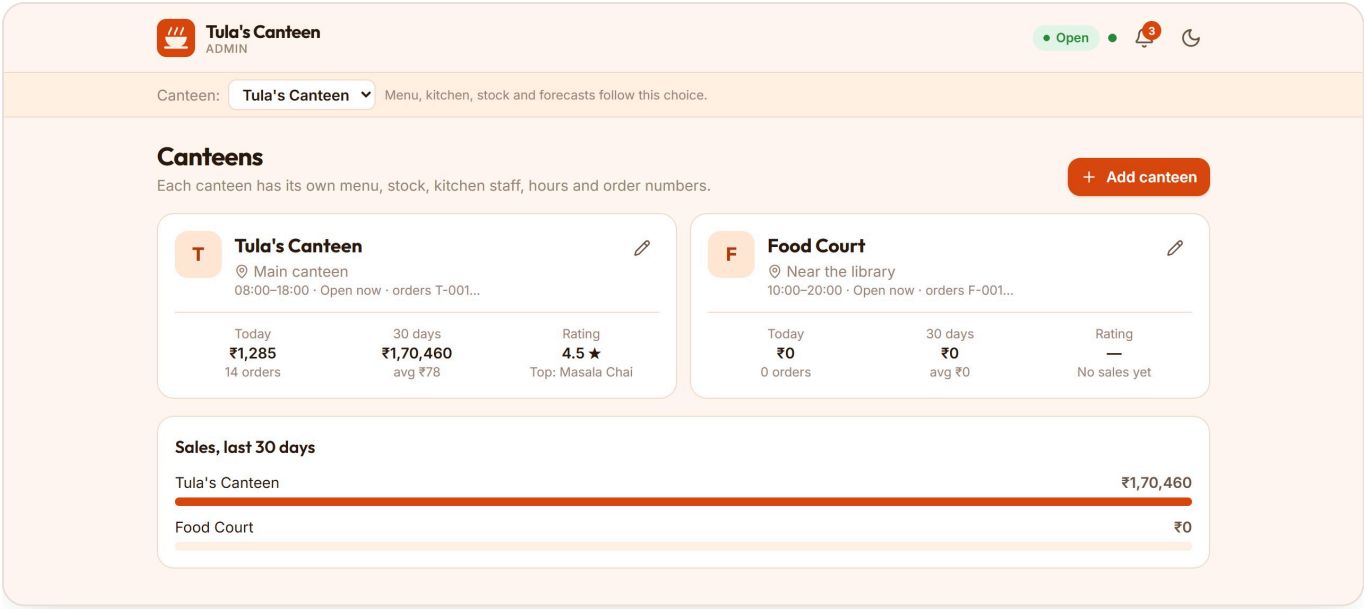
Reviews, forecasts and canteens



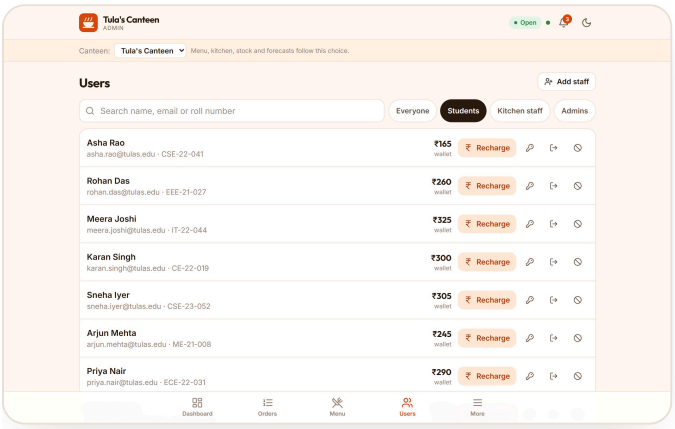
Dish reviews. Words that look like a link, phone number, email or abuse are held until an admin checks them. Hiding a review hides its words only; its stars still count.



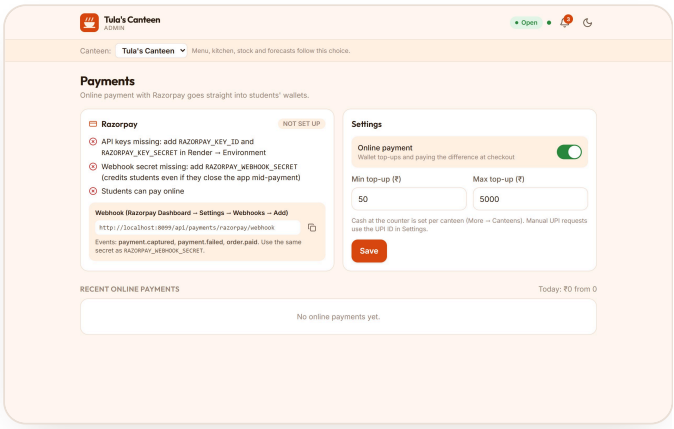
Forecast & insights. Plates to prepare per dish for any of the next 7 days with a likely range, the ingredients needed, an accuracy check against the last 14 days, the peak-hours heatmap and best and least sellers. One click uses the forecast as tomorrow's daily plates.



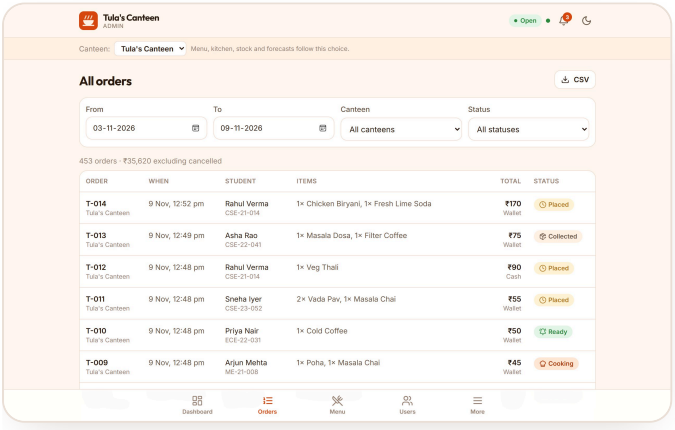
Canteens. Each canteen has its own menu, stock, kitchen staff, hours, rules and order prefix (`T-017` at the main canteen, `F-017` at the Food Court), compared side by side.



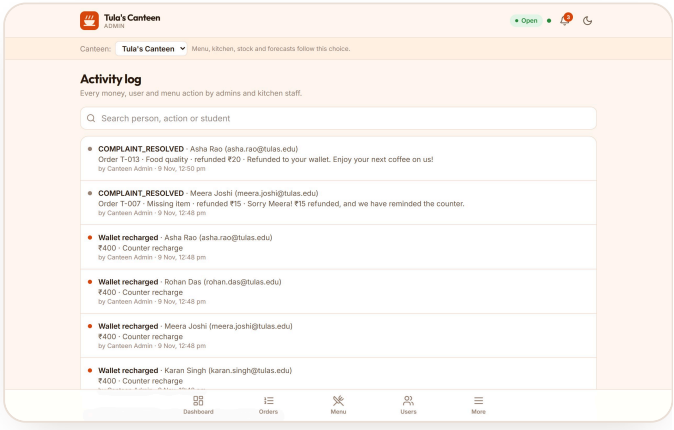
Users. Search, recharge wallets, block, reset passwords, create kitchen logins.



Payments. Online payment on/off, limits, test/live mode and every Razorpay payment.



Orders. Every order with date and status filters and a CSV export for Excel.



Activity log. Who recharged, approved, refunded, blocked or changed the menu, and when.

Admins also manage the menu (with a one-click standard menu of 43 everyday dishes to start from), approve UPI top-ups, post announcements shown at the top of every student's screen, and set each canteen's hours, order limits and no-show hold time.

06 · HOW THE SMART PARTS WORK

Inside the intelligence

There is no black-box AI here. Every recommendation and forecast is calculated from the canteen's own orders with simple, explainable statistics, and every result comes with a reason a person can check.

1 · Recommendations: basket analysis

Over the last 60 days of orders, the server counts how often each dish is ordered and how often two dishes appear in the same order. For a pair of dishes A and B:

```
confidence(A→B) = orders with A and B ÷ orders with A
lift(A→B)       = confidence(A→B) ÷ share of all orders that contain B
```

Confidence becomes the sentence students see: *"60% of people who order Masala Dosa add Filter Coffee."* A lift above 1 means the pair is bought together more often than chance. A pair must be seen at least twice before it is trusted.

The personal feed scores each available dish for a student by combining the basket rules with their own taste:

```
score = 3.0 × goes-with-what-you-order + 1.0 × favourite-category
      + 0.8 × popular-this-week + 0.5 × time-of-day + 0.3 × rating
```

The student's own orders count more when they are recent (a 14-day half-life), and a student who ordered only veg dishes gets only veg suggestions. Each card shows its reason: "Goes well with Veg Thali", "Because you like breakfast", "Popular this week". The same rules power cart add-ons and alternatives for sold-out dishes.

2 · Demand forecast: how many plates tomorrow?

```
weekday = 40% × last week + 30% × 2 weeks ago + 20% × 3 weeks ago + 10% × 4 weeks ago (same
weekday)
trend   = exponentially weighted average of the last 7 open days (α = 0.3)
forecast = (60% × weekday + 40% × trend) × 1.10 safety margin
```

Days the canteen was closed are ignored, and a weekday it is always closed on forecasts zero. The likely range (about 80%) comes from how wrong past forecasts were. Multiplying by each dish's per-plate recipe gives the shopping list: *17 plates of Rajma Chawal → 1.19 kg rajma.*

Accuracy check. To prove the method, the server replays the last 14 open days using only the data that existed before each day, and compares the forecast with what actually sold (average miss and MAPE).

3 · Reward points

Rule	How it works
Earning	1 point per ₹10 when an order is collected. Silver (₹2,000+ spent) earns 1.25×, Gold (₹5,000+) 1.5×.
Spending	In steps of 10 points; 10 points = ₹5; at most half of an order.

Bonuses	50 points once a year on the student's birthday; 50 points each for a student and the friend they invite, after the friend's first collected order.
Safety	Points are a ledger with atomic updates, like the wallet; points used on a cancelled order come back.

4 · Review moderation

A review's words are checked as they are saved. Text that contains a link, an email address, a phone number or a word from a short list of strong abusive words (English and common Hindi transliterations) is **held** for an admin instead of being shown. Everything else appears immediately.

- Each (order, dish) counts once in a dish's rating. Rating the whole order gives every dish the same stars; a dish review **replaces** that share with its own stars instead of adding a second vote.
- Hiding a review hides its words, not its stars, so moderation can't be used to push a rating up or down.
- If an admin hides a review and the student edits the words, it goes back for a check rather than straight back up.

5 · Live status and wait time

The pickup pass shows how many orders are ahead in the kitchen and an estimated wait, based on the longest prep time in each order. It updates through Server-Sent Events whenever any order in that canteen changes, so students never need to refresh.

07 · SYSTEM ARCHITECTURE

How the pieces fit together

A single Spring Boot application serves both the React app and the API from one Docker image. The browser talks to it over HTTPS with a JWT on every request, and keeps one Server-Sent Events connection open for live updates.

Browser

React 19 app
Student, kitchen and admin screens

EventSource (SSE)
Live orders, wallet, notifications

Service worker
Phone alerts via Web Push

Razorpay Checkout
Online payments

Spring Boot server

Security
JWT filter, role rules per path, login guard

Controllers (9)
Auth, student, kitchen, admin, feedback, insights, payments, notifications, public

Services
Orders, wallet, loyalty, menu, forecast, recommendations, complaints, reviews

LiveHub + PushSender
Events after commit; encrypted Web Push

Scheduled jobs
No-shows, stale orders, 5 AM stock reset, birthdays

Outside services

PostgreSQL (Neon)
All data, 22 tables

Razorpay
Orders, signature checks, webhook

Push services
Google, Mozilla, Apple, Microsoft

Render
Docker hosting, HTTPS, health checks

Design choices

- **Layered backend.** Controllers handle HTTP and validation, services hold the business rules and transactions, repositories talk to the database. Money and stock changes live in one place each (`WalletService`, the menu repository).
- **Live updates after commit.** Events are sent only once the database change is committed, so a screen never shows something that was then rolled back.
- **One deployable.** The React build is bundled into the jar; unknown paths like `/orders/12` fall back to `index.html` so the React router handles them.
- **Stateless web layer.** The JWT carries the user, so more server instances could be added behind a load balancer (live events would then need a shared message bus).
- **Database portability.** The same code runs on H2 (zero-setup local runs and tests), MySQL and PostgreSQL.

08 · THE LIFE OF AN ORDER

What happens when you tap "Place order"

STEP 1
Order number
The day's counter row is created in its own short transaction.

→

STEP 2
One transaction
Rules, plates, payment, points and the order: all or nothing.

→

STEP 3
After commit
Live event to the kitchen board; the student's pass opens.

→

STEP 4
Kitchen
Placed → Cooking → Ready, each change pushed to the student.

Inside the transaction

1. **Idempotency check**
   The app sends an `Idempotency-Key` with every checkout. A double tap or a retry on bad Wi-Fi returns the first order instead of charging twice.

2. **Rules**
   The canteen must be open and accepting orders; the pickup slot must be in opening hours; limits on orders in progress and plates of one dish apply; cash must be allowed for cash orders.

3. **Reserve plates**
   `UPDATE … SET stock = stock - qty WHERE id = ? AND stock ≥ qty`. If no row changes, the dish just sold out and the whole order rolls back. Two students can never buy the last plate.

4. **Take payment**
   The wallet debit is also a conditional `UPDATE` (balance ≥ amount), followed by a ledger row with the balance afterwards. A wallet can never go negative.

5. **Points and save**
   Reward points are spent the same way, a 4-digit code is generated with a secure random generator, and the order is saved.

Status changes

Status	Who / what changes it
Placed	The student; can still cancel with an instant refund.
Cooking	Kitchen taps Start cooking (from here the student can no longer cancel).
Ready	Kitchen taps Mark ready: notification, vibration and a push alert.
Collected	Correct pickup code at the counter. Optimistic locking (`@Version`) stops two staff phones handing over the same order twice. Points are earned.
Not collected	A job every minute: ready food not collected within the hold time. No refund (it was cooked); plates go back on sale.
Cancelled	The student before cooking, the kitchen with a reason, or a job for unfinished orders from earlier days. Wallet payments and points are refunded.

09 · TECHNOLOGY STACK

Tools and why they were chosen

Technology	Layer	Why it is used
Java 21	Language	Modern, fast and strongly typed; records keep DTOs short.
Spring Boot 3.5	Backend	Web, security, data, validation and scheduling in one well-known framework.
Spring Security + JWT (jjwt)	Auth	Stateless logins and role rules on every path.
Spring Data JPA / Hibernate	Data access	Entities and repositories, with hand-written atomic `UPDATE`s where it matters.
PostgreSQL on Neon	Database	Managed, serverless PostgreSQL with a free tier; same region as the server.
H2 and MySQL 8	Database	H2 for zero-setup local runs and fast tests; MySQL via Docker Compose.
Server-Sent Events	Live updates	Simple one-way push over HTTP; perfect for status updates, no extra server.
Web Push (web-push, Bouncy Castle)	Phone alerts	Standard encrypted push (RFC 8291, VAPID RFC 8292) that works with the site closed.
Razorpay	Payments	UPI, cards, netbanking and wallets for Indian students, with a test mode.
springdoc OpenAPI	API docs	Interactive Swagger UI generated from the code.
React 19 + Vite	Frontend	Fast, component-based UI with instant reloads in development.
Tailwind CSS 4	Styling	A consistent design system with light and dark mode and a mobile-first layout.
React Router, Lucide, qrcode	Frontend	Pages, icons, and the QR code on the pickup pass.
JUnit 5, MockMvc, embedded PostgreSQL	Testing	End-to-end API tests on H2 and a real PostgreSQL server.
Docker, Render, Neon	Deployment	One image, deployed automatically on every push, at zero cost.

10 · DATABASE DESIGN

Where the data lives

Twenty-two tables, created and updated by Hibernate from the entity classes. Money, stock and counters are changed with single atomic SQL updates, and the most important rules are also enforced by unique constraints in the database itself.

Area	Tables	Notes
People	`users`, `user_favourites`, `push_subscriptions`	Role, canteen (for staff), wallet balance, points, tier, referral code; BCrypt password hash.
Canteens and menu	`canteens`, `canteen_settings`, `menu_items`, `dish_ingredients`	Hours, limits, status and prefix per canteen; dishes with stock today and default stock; per-plate recipes.
Orders	`orders`, `order_items`, `daily_counters`, `order_counters`	Unique (day, code) and unique (user, idempotency key). Items copy name and price, so later menu edits don't rewrite history.
Money	`wallet_txns`, `topup_requests`, `online_payments`, `loyalty_txns`	Every movement with the balance afterwards. A UPI reference and a Razorpay payment can each be used once.
Feedback	`dish_reviews`, `complaints`, `complaint_messages`	One review per (order, dish); one complaint per order; version column against double refunds.
Operations	`notifications`, `announcements`, `audit_log`, `app_secrets`	The bell, banners, the admin activity log, and generated VAPID keys for push.

Integrity rules worth mentioning

- **Soft delete for dishes.** Removed dishes are archived, so old orders and reports still make sense.
- **No lost updates.** Users and dishes use dynamic updates: saving an unrelated change (like blocking a user) only writes the changed columns, so it can't overwrite a balance or stock changed by someone else at the same moment.
- **Optimistic locking** on orders, top-up requests and complaints: two people acting on the same record at once can't both succeed.
- **Indexes** on the queries that run all day: orders by day and status, by user and date; notifications by user; complaints by status.

How the frontend talks to the backend

REST: 101 endpoints in 9 controllers

Area	Count	Examples
Auth	5	`POST /api/auth/register`, `/login`, `GET/PUT /api/auth/me`, `POST /api/auth/password`
Public	6	`GET /api/menu`, `/api/canteens`, `/api/announcements`, `/api/live` (SSE), `/api/health`
Student	16	`POST /api/orders`, `/orders/{id}/cancel`, `/orders/{id}/rate`, `GET /api/wallet`, `/api/rewards`, `/api/recommendations`
Kitchen	7	`GET /api/kitchen/queue`, `POST /orders/{id}/status`, `/verify`, `PUT /menu/{id}/stock`, `/status`
Admin	29	Dashboard, users, recharge, staff, top-ups, menu, orders and CSV, audit, announcements, settings, canteens
Feedback	15	`POST /api/orders/{id}/complaint`, `/reviews`, `GET /api/reviews/dish/{id}`, `/api/admin/complaints/{id}/resolve`
Insights	7	`/api/admin/insights/forecast`, `/accuracy`, `/peaks`, `/sellers`, recipes
Payments	9	Razorpay order, verify, webhook, admin settings and payment list
Notifications	7	The bell, mark read, push subscribe / unsubscribe, VAPID key, test alert

Every request body is validated (sizes, ranges, formats), and errors come back as a readable message the app shows as is. Interactive docs are at `/swagger-ui.html`.

Live updates: one SSE stream per tab

After login the app opens `GET /api/live`. The server pushes named events, and each screen re-fetches what it shows when a relevant event arrives:

Event	Sent to	Used by
`order`	The student and the canteen's staff	Pickup pass, orders list, kitchen board, dashboard
`wallet`, `points`	The student	Wallet balance, cart, rewards
`menu`, `canteen`	Everyone	Stock badges, open / busy / closed
`notification`	One user	The bell's unread count
`topup`, `complaint`	Student and admins	Approvals, complaint threads and badges

A heartbeat every 25 seconds keeps connections alive through proxies, and the app reconnects automatically after a drop.

12 · SECURITY

Keeping money and data safe

Authentication with JWT

After login the server issues a signed token (72 hours by default). Every request carries it; the server checks the signature, the expiry and that the user is still active.

Password storage

Only BCrypt hashes are stored. Changing or resetting a password logs out every other device.

Role rules on the server

Each path is restricted by role (`/api/admin/**` admins only, `/api/kitchen/**` staff and admins), and staff are also checked against their own canteen.

Login protection

5 wrong passwords lock that account for 15 minutes; failed logins and sign-ups are also limited per network.

Secure defaults

No well-known passwords in production: a random admin password and JWT secret are generated if none are set. Demo logins exist only in local demo mode.

Payments you can trust

Amounts are fixed on the server; every Razorpay payment is verified with an HMAC-SHA256 signature (constant-time compare) and credited exactly once. Keys live only in environment variables.

Push without SSRF

Push subscriptions are accepted only for the official push services (Google, Mozilla, Apple, Microsoft), so the server can't be tricked into calling other addresses.

Audit trail

Recharges, approvals, refunds, blocks, password resets, menu changes and complaint outcomes are logged with who did it and when.

Safe inputs and errors

Validation on every request, escaped search patterns, readable error messages; stack traces stay in the server log.

Hand-over code

The 4-digit code is shown only to the student who owns the order; 5 wrong attempts lock the order at the counter.

Real problems found and solved

These are real issues found while taking the project to production. They make excellent talking points because they show debugging skill, not just feature building.

Problem	Root cause	Solution
Admin Users, Orders and Activity pages failed only on the live site	Optional filters written as `:q is null or …`: PostgreSQL rejects an untyped null ("function lower(bytea) does not exist"); H2 accepted it	Queries without nullable parameters (blank search → `'%'`, missing status → all), plus tests that load every page on PostgreSQL
Checkout could stall in a lunch rush	Each order held two database connections (the day's counter in a nested transaction); a burst could exhaust Neon's 5-connection pool	Create the counter first in its own short transaction; a concurrency test fires simultaneous orders
First requests after a restart sometimes failed	Concurrent requests raced to insert the settings row	Create it during start-up, before the web server opens
A wallet balance could be silently overwritten	Saving an unrelated change (e.g. blocking a user) wrote back every column, including a balance changed meanwhile	Dynamic updates (only changed columns) plus atomic SQL for money, points and stock
Two students, one last plate	Read-then-write stock check	Conditional `UPDATE … WHERE stock ≥ qty`; the loser gets "sold out"
Double charge on a double tap	Retried requests looked like new orders	Idempotency key with a unique constraint per user
Two phones handing over one order	Both read "Ready" at the same time	Optimistic locking: the second save fails cleanly
A Razorpay payment credited twice	The app's confirmation and the webhook can arrive together	A unique payment record; whichever arrives first credits, the other is a no-op
A complaint refunded twice	Two admins resolving at the same moment	Version check before any money moves
Dish ratings counted twice	Rating the order and reviewing a dish both added a vote	A dish review replaces the order's stars for that dish; atomic rating updates
Dropped database connections	Neon closes idle connections	Pool connections recycled after 4 minutes, idle ones after 2

14 · TESTING AND QUALITY

Proof that it works

60
automated tests in the build

2
databases every flow test runs on: H2 and a real PostgreSQL

10
steps in the recorded browser walkthrough of the demo video

What is tested

- **The whole order flow** through the real API: order, cook, ready, hand over with the code, wrong codes and lockout, cancel and refund.
- **Concurrency:** simultaneous orders get unique numbers; the first order of the day survives a race; a burst of orders doesn't exhaust the connection pool.
- **Money:** wallet payments and refunds, UPI top-ups approved once, rejected requests don't pay, reward points earned, used and returned.
- **Online payment** against a fake Razorpay API: signatures, the webhook, and crediting exactly once.
- **Multi-canteen:** each kitchen sees and serves only its own orders.
- **Smart parts:** recommendations learn what is ordered together; the forecast follows the weekly pattern and scales ingredients.
- **Feedback:** complaints refunded once and capped at what was paid; cash no-shows can't be refunded; reviews count once and suspicious words wait for an admin.
- **Security:** unauthenticated and wrong-role requests are refused; account lockout; password changes log out other devices; push only to real push services.
- **Every page loads** for students, kitchen and admins, with and without filters.

> **Why run the same tests on PostgreSQL?** H2 is fast and needs no setup, but it accepts SQL that PostgreSQL rejects. A real bug on the live site (section 13) was exactly that. Every flow test now also runs against an embedded PostgreSQL server, the same database Neon runs.

Beyond the automated tests, each feature was checked in a real browser: a script drives headless Edge through the app as a student, a cook and an admin, and the captioned demo video was recorded by that same script.

15 · DEPLOYMENT AND OPERATIONS

How it runs on the internet

1 · GITHUB
Code is pushed to the **main** branch.

→

2 · RENDER BUILD
The Dockerfile builds the React app and the Spring Boot jar into one image.

→

3 · HEALTH CHECK
The new version goes live only once `/api/health` answers.

→

4 · LIVE
Served over HTTPS; tables are created or updated on start.

The setup is described in `render.yaml` (infrastructure as code). `/api/health` also reports the commit that is running, which is how to confirm a deploy finished.

Configuration (environment variables)

Variable	Purpose
`DATABASE_URL`	The Neon connection string, pasted as is (converted to JDBC settings automatically)
`ADMIN_EMAIL` · `ADMIN_PASSWORD`	The first admin account, created on first start
`JWT_SECRET`	Signing key for login tokens, generated by Render
`RAZORPAY_KEY_ID` · `RAZORPAY_KEY_SECRET`	Online payment: `rzp_test_…` for test mode, `rzp_live_…` for real money; unset = off
`RAZORPAY_WEBHOOK_SECRET`	Verifies Razorpay's webhook
`SEED_DEMO` · `TZ`	Demo data off in production; Indian time zone

Free-tier behaviour to know

- **Sleeping server:** Render's free service sleeps after 15 minutes without visitors and takes about a minute to wake. Open the site before a demo.
- **Database:** Neon pauses an idle database and wakes it in about a second; the connection pool is tuned for this.
- **Memory:** JVM flags are tuned to fit the 512 MB free instance.

Running it locally

With JDK 21, Maven and Node.js 20: run `mvn spring-boot:run` in `backend` and `npm install` then `npm run dev` in `frontend`, and open `http://localhost:5173`. Local runs start in demo mode with two canteens, four weeks of sample sales and demo logins (student, kitchen and admin). `docker compose up` runs it with MySQL instead.

Your presentation playbook

The 30-second pitch (learn it by heart)

> "Every college canteen has the same lunch-hour queue. Tula's Canteen removes it: students order and pay from their phone, the kitchen cooks from a live board, and food is handed over in seconds with a pickup code. Admins see what sells, get a forecast of how many plates to make, and handle complaints with refunds in one place. It's live on the web, handles a lunch rush safely, is tested on a real PostgreSQL database, and runs at zero hosting cost."

Structure for a 5–10 minute talk

1. **Problem (1 min)**
   The queue, the cash counter, guesswork cooking (section 2).
2. **Solution (30 s)**
   The pitch above.
3. **Live demo (5 min)**
   The script on the next page.
4. **How it works (2 min)**
   The order transaction, live updates and the forecast (sections 6–8).
5. **Engineering (1 min)**
   Two or three stories from section 13.
6. **Close (30 s)**
   Impact and what comes next.

Adjust to your audience

Audience	Focus on	Keep short
Canteen owner, college staff	Shorter queues, cashless payments, fewer wasted plates, complaints handled	Technology
Senior engineer, interviewer	The order transaction, atomic updates, idempotency, locking, SSE, PostgreSQL bug story	The basic feature tour
Friends, students	The story and the live demo on a phone	Internals
Judges, events	Problem → demo → architecture → impact in about 5 minutes	Long tangents

Before every demo

- **Wake the server:** open the site 5 minutes early (the free server sleeps).
- **Prepare accounts:** a student with money in the wallet and a kitchen login, in two browser windows (or a phone and a laptop).

- **Check the canteen is Open** on the kitchen board.
- **Backup plan:** the demo video and the screenshots in this guide, if the internet fails. Never show passwords on screen.

Live demo script (about 7 minutes)

1 · Sign in as a student — 20 s

Log in on a phone or a narrow window.

"Each student has their own secure account and a campus wallet."

2 · The menu — 45 s

Switch canteens, filter veg, tap a rating to show reviews, point at "only N left".

"Stock is live: when the last plate goes, it shows sold out everywhere instantly."

3 · Cart and checkout — 1 min

Add a dish, accept an add-on suggestion, choose a pickup time, use points, place the order.

"These suggestions come from what students actually order together."

4 · Pickup pass — 30 s

Show the code, the QR and the queue position.

"No more shouting order numbers: this code is my receipt."

5 · Kitchen board ⭐ — 1 min

In the kitchen window, the new order appeared with a chime. Start cooking, then Mark ready, and show the student's pass updating by itself.

"Nobody refreshed anything: it's pushed live to both screens."

6 · Hand-over — 30 s

Type the order number and code (or scan the QR).

"Wrong codes are counted; after five the order locks."

7 · Feedback — 45 s

As the student, rate a dish and report a problem.

"Complaints don't get lost in the corridor any more."

8 · Admin: complaint and refund — 1 min

Reply in the thread, then resolve with a refund for one dish; show the wallet history.

"The refund can never be more than what was paid, and it can only happen once."

9 · Dashboard and forecast — 1 min

Sales, busy hours, top dishes; then tomorrow's plates and ingredients.

"The kitchen knows how many plates to make, and what to buy."

> **Closing line:** "In short: order ahead, skip the queue, and run the canteen on data. It's live, it's safe under a lunch rush, and it runs at zero hosting cost. Happy to go deeper into anything."

17 · QUESTIONS YOU WILL BE ASKED

Prepared answers

What stops two students buying the last plate?

Plates are reserved with a single conditional SQL update that only succeeds if enough plates are left. The database does the check and the change in one step, so one student gets the plate and the other sees "sold out". The whole order then rolls back for the second student.

What if the Wi-Fi drops while I pay?

Every checkout carries an idempotency key. If the app retries, the server finds the first order and returns it instead of charging again. For Razorpay, a signed webhook credits the wallet even if the app is closed mid-payment, and each payment is credited exactly once.

Why Server-Sent Events and not WebSockets?

Updates only flow from server to browser, and the browser sends its actions as normal API calls. SSE is plain HTTP, reconnects automatically and needs no extra infrastructure. WebSockets would add complexity without benefit here.

Is the "AI" really AI?

It's statistics, on purpose: basket analysis (confidence and lift) for recommendations and weighted averages with a trend for forecasts. They need little data, are fast and every result can be explained, which matters more for a canteen than a black box. The accuracy check proves how good the forecast is.

How do you stop fake complaints or refund abuse?

Only the student who placed an order can report it, within 48 hours and once per order, with at most three open at a time. A human decides every refund, the refund is capped at what was actually paid (a cash no-show paid nothing), and every outcome is in the activity log.

Can't a canteen just hide bad reviews?

Hiding a review hides its words only; its stars still count towards the rating. Moderation is for links, phone numbers and abuse, not for opinions.

Does it scale beyond one canteen?

Yes: canteens are data, each with its own menu, kitchen staff, stock and order numbers, and one wallet works across all of them. The web layer is stateless, so more instances can be added; live updates would then move to a shared message bus such as Redis.

Why PostgreSQL, and why also H2 and MySQL?

PostgreSQL on Neon is the production database (managed and free to start). H2 makes local runs and tests instant with no setup, and MySQL is supported through Docker Compose for colleges that already run it. Running every test on PostgreSQL too catches differences between them.

How much does it cost to run?

Nothing today: Render, Neon and Razorpay's test mode are free. For a real canteen, roughly $7/month for an always-on server, plus Razorpay's per-transaction fee on online payments.

Did you build all of it yourself?

Answer honestly and specifically: explain how you built it and which tools you used, and be ready to walk through any part of the code (the order transaction, the forecast, the security rules). Being able to explain every decision earns more respect than any claim.

18 · FUTURE ROADMAP

Where it could go next

1

Live payments

Complete Razorpay KYC, switch to live keys and add automatic refunds to the original payment method.

2

Install as an app

A full Progressive Web App with an install prompt and offline menu, or a native app built on the same API.

3

Meal plans and passes

Monthly mess plans and hostel meal passes, deducted per meal.

4

Inventory and purchases

Raw-material stock used up from recipes on every order, supplier purchases and profit per dish.

5

Coupons and combos

Promo codes with limits and expiry, and combo meals at a bundle price.

6

Receipts and GST

PDF receipts and GST invoices per order, and monthly wallet statements.

7

Email and WhatsApp

Order-ready and refund messages on channels students already use.

8

Smarter forecasts

Use the academic calendar (exams, holidays, events) and weather as extra signals.

Terms explained simply

JWT	JSON Web Token: a signed pass the server gives you at login and checks on every request.
BCrypt	A deliberately slow way of storing passwords so they are very hard to crack.
Transaction	A group of database changes that all succeed together or are all undone.
Atomic update	A single SQL statement that checks and changes a value in one step, so nobody can change it in between.
Idempotency key	A unique id sent with a request so that repeating it doesn't repeat its effect (no double orders).
Optimistic locking	Each record has a version number; if two people save the same version, the second save fails instead of overwriting.
Server-Sent Events (SSE)	A long-lived HTTP connection over which the server pushes updates to the browser.
Web Push / VAPID	The browser standard for phone and desktop alerts, even with the site closed; VAPID keys prove who sent them.
Webhook	A call from another service (here Razorpay) to our server when something happens, like a completed payment.
HMAC signature	A code computed with a shared secret that proves a message really came from Razorpay and wasn't changed.
Basket analysis	Finding which items are bought together, measured by confidence and lift.
MAPE	Mean absolute percentage error: on average, how far the forecast was from what actually sold.
No-show	Food that was cooked but not collected within the canteen's hold time.
SPA	Single-page application: the React app loads once and then changes screens without reloading.
Neon / Render	The hosted PostgreSQL database and the hosting platform that runs the Docker image.
H2	A small database that runs inside the app, used for local runs and fast tests.

20 · CREDITS

About this project

Developer	Abhishek Raj
Repository	github.com/Abhishek360007/Canteen-Management-System-
Live site	tulas-canteen.onrender.com
Demo video	abhishek360007.github.io/Canteen-Management-System-/demo/
License	MIT

Built in stages

- Ordering ahead, the live kitchen board, pickup codes, the wallet and the admin dashboard
- Secure defaults, PostgreSQL on Neon and deployment on Render
- The standard menu, UPI top-ups, the activity log and login protection
- Smart recommendations and reward points
- Sales and demand forecasting with ingredient planning
- Notifications: the in-app bell and browser push
- Multiple canteens with their own menus and kitchens
- Online payment with Razorpay
- Feedback and complaints: dish reviews with moderation, complaint threads and refunds



Tula's Canteen

Order ahead. Skip the queue.